

# Linear Regression House Price Predictor

## Task 1 Machine Learning Report

### Project Objective

The objective of this task was to build and evaluate a Linear Regression model for house price prediction using the California Housing dataset.

The project introduces the complete machine learning workflow, starting from data loading and ending with evaluation, visualization, model saving, and reporting.

### Work Completed

- Loaded the California Housing dataset from an uploaded CSV file.
- Explored the data using summary statistics, missing value checks, distributions, and correlations.
- Preprocessed the dataset by handling missing values, engineering features, and encoding categorical columns.
- Split the data into training and testing sets.
- Trained a LinearRegression model using scikit-learn.
- Evaluated the model using MAE, RMSE, and R2 score.
- Created prediction and residual plots to understand model performance.
- Saved the trained model for future prediction use.

### Deliverables

- Jupyter notebook with code, EDA, model training, plots, and comments.
- Short PDF report summarizing dataset, EDA, model, metrics, and improvement ideas.
- Optional saved model output from the notebook.

# Dataset and Preprocessing

## Dataset Used

The dataset used in the executed notebook was the California Housing dataset uploaded as housing.csv. This dataset contains information about housing blocks in California and is commonly used for regression practice.

The prediction target was MedHouseVal, which represents the median house value. The target was scaled into 100,000 dollar units so that the output remains consistent with the standard California Housing dataset format.

## Dataset Shape

- Total rows: 20,640
- Columns after preprocessing: 13
- Training rows: 16,512
- Testing rows: 4,128

## Preprocessing Steps

The original CSV contained numeric variables such as longitude, latitude, housing age, total rooms, total bedrooms, population, households, median income, and median house value. It also included the categorical variable ocean\_proximity.

Missing numeric values were checked and handled using median filling. This approach is simple and avoids losing rows while keeping the preprocessing easy to understand.

Feature engineering was applied to make the raw count variables more useful. Total rooms, total bedrooms, and households were converted into average room, average bedroom, and average occupancy features.

The ocean\_proximity column was converted into numeric one-hot encoded columns so that the Linear Regression model could use it.

# Exploratory Data Analysis

## EDA Purpose

Exploratory Data Analysis was performed to understand the dataset before model training. This step helped identify the feature ranges, distributions, missing values, and relationships between features and the target variable.

## Checks Performed

- Used `df.info()` to inspect column names, data types, and non-null counts.
- Used missing value counts to confirm whether any columns required imputation.
- Used descriptive statistics to understand mean, minimum, maximum, and quartile values.
- Plotted feature distributions to check skewness and unusual ranges.
- Created a correlation heatmap to compare relationships between features.
- Created a location-based scatter plot to visualize California housing prices geographically.

## Main EDA Findings

Median income had the strongest positive correlation with median house value. This means areas with higher median income generally had higher house prices.

Ocean proximity also mattered. Inland areas had a strong negative relationship with house value, while near-bay and near-ocean locations tended to be more valuable.

Latitude and longitude were useful because they captured geographic patterns in California housing prices. The location plot showed that house values were not evenly distributed across the state.

Some variables were skewed, especially population and occupancy-related features. This suggests that future versions of the model could benefit from transformations or outlier handling.

# Model Training Methodology

## Feature Selection

The target variable was MedHouseVal. All remaining numeric columns after preprocessing were used as input features. These included income, location, housing age, population, engineered household features, and encoded ocean-proximity categories.

## Train-Test Split

The dataset was split into training and testing sets using an 80 percent training and 20 percent testing split. The `random_state` value was fixed so the results could be reproduced.

The model was trained on 16,512 rows and tested on 4,128 rows. Keeping a separate test set allowed the final metrics to measure performance on unseen data instead of only measuring how well the model memorized the training data.

## Model Used

The model used was LinearRegression from scikit-learn. Linear Regression is a good baseline because it is fast, simple, and interpretable. It tries to learn a straight-line relationship between the input features and the target value.

## Why Linear Regression Was Suitable Here

- It satisfies the task requirement directly.
- It provides an easy-to-explain starting point for regression.
- Coefficients can be inspected to understand feature influence.
- It gives a baseline that can later be compared with stronger models.

## Model Output

After training, the model was saved as `california_linear_regression_model.pkl`. The notebook also included a small prediction function for estimating house value from new input values.

# Evaluation Results

## Metrics Used

Three evaluation metrics were used: Mean Absolute Error, Root Mean Squared Error, and R2 score.

MAE measures the average absolute difference between actual and predicted house values.

RMSE is similar but penalizes larger errors more strongly. R2 score measures how much of the target variation is explained by the model.

## Final Results

- MAE: 0.538797, which is about 53,880 dollars average absolute error.
- RMSE: 0.812622, which is about 81,262 dollars with larger errors penalized more heavily.
- R2 Score: 0.496070, meaning the model explains about 49.6 percent of the test-set variance.

## Interpretation

The Linear Regression model was able to capture some important patterns in the dataset, especially the relationship between income, location, and house value. However, the R2 score shows that the model explains only about half of the variation in house prices.

This result is reasonable for a baseline model. Housing prices depend on many nonlinear relationships and interactions, so a simple linear model cannot capture every pattern.

## Visual Evaluation

The predicted vs actual scatter plot showed that predictions followed the overall direction of the actual values, but there was still noticeable spread. This means the model was not perfectly accurate for individual houses.

The residual plot showed remaining patterns in the errors. Ideally, residuals should look random, so visible patterns suggest that the model is underfitting some parts of the data.

# Conclusion and Improvement Ideas

## Conclusion

This task successfully completed the full beginner-friendly machine learning lifecycle. The dataset was loaded, explored, preprocessed, split into train and test sets, used to train a Linear Regression model, evaluated using standard regression metrics, and documented in a notebook and report.

The model provides a useful baseline for California house price prediction. Median income and location-based features were the most important signals. The results show that Linear Regression is understandable and easy to use, but it has limitations for a complex real-world problem like housing prices.

## Limitations

- Linear Regression assumes a mostly linear relationship between features and target.
- Housing prices are affected by nonlinear patterns, outliers, and feature interactions.
- Some important real-world factors, such as school quality, exact neighborhood, and market conditions, were not included in the dataset.
- The model may underfit because it is too simple for the full complexity of housing prices.

## Future Improvements

- Apply feature scaling and compare results.
- Try Ridge Regression and Lasso Regression for regularized linear models.
- Use polynomial features or interaction terms to capture nonlinear patterns.
- Compare with Random Forest, Gradient Boosting, or XGBoost models.
- Use cross-validation for more reliable performance estimation.
- Improve feature engineering with stronger location-based features.

Overall, this project is a clean portfolio-style introduction to regression modeling and the machine learning workflow.